CS5284 : Graph Machine Learning

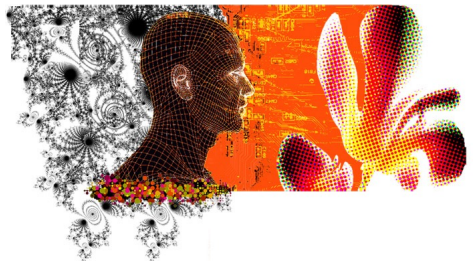
Lecture 7 : Shallow Graph Feature Learning

Semester 1 2025/26

Xavier Bresson

https://x.com/xbresson

Department of Computer Science
National University of Singapore (NUS)



Course lectures

- Introduction to Graph Machine Learning
- Part 1: GML without feature learning (before 2014)
  - Introduction to Graph Science
  - Graph Analysis Techniques without Feature Learning
    - Graph clustering
    - Graph SVM
    - Recommendation on graphs
    - Graph-based visualization
- Part 2 : GML with shallow feature learning (2014-2016)
  - Shallow graph feature learning
- Part 3 : GML with deep feature learning, a.k.a. GNNs (after 2016)
  - Graph Convolutional Networks (spectral and spatial)
  - Weisfeiler-Lehman GNNs
  - Graph Transformer & Graph ViT
  - Graph generation & molecular science
  - Material design
  - Integrating GNNs and LLMs
  - Deep Learning for Combinatorial optimization

Outline

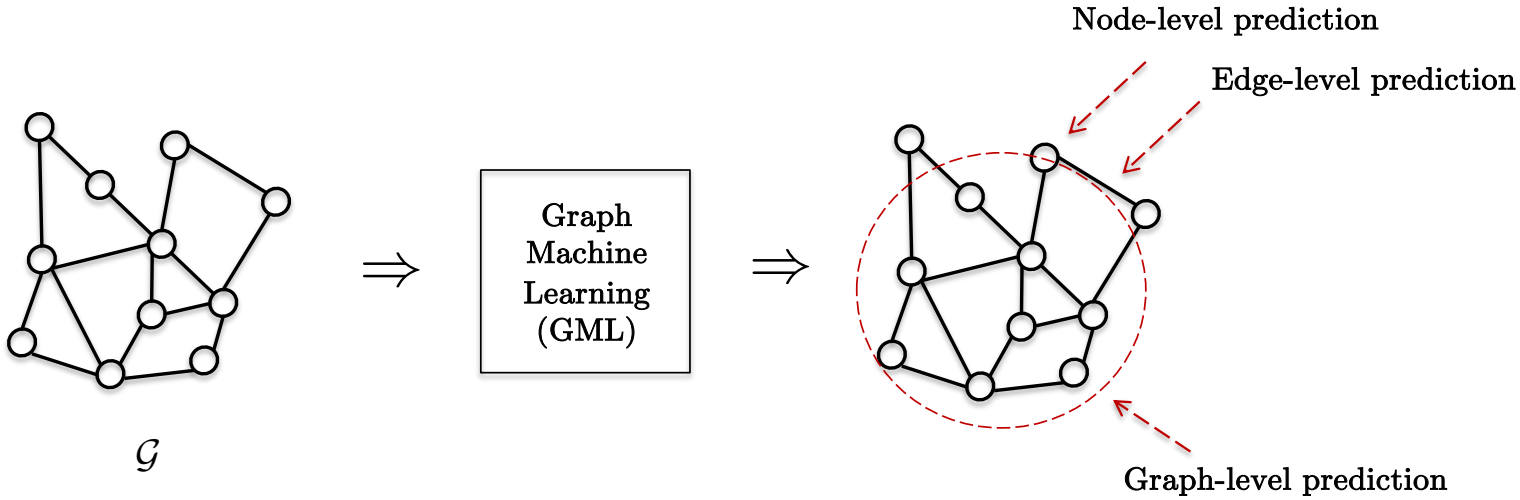
- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

Outline

- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

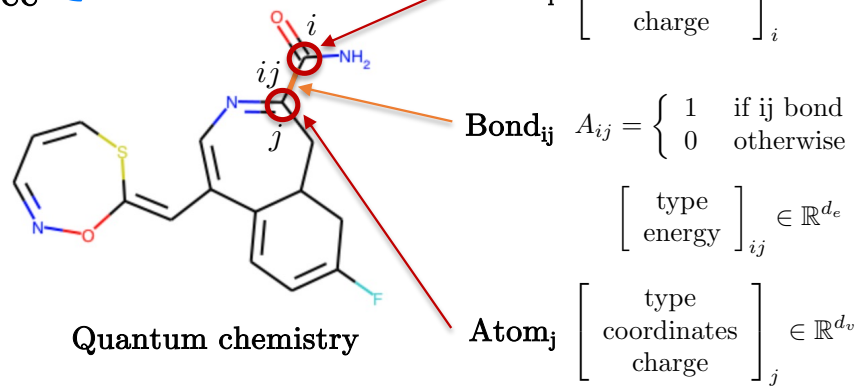
Graph prediction tasks

- There are three fundamental prediction tasks with graphs :
  - Node-level prediction, e.g. identify a fraudster in financial networks.
  - Edge/link-level prediction, e.g. detecting a fraudulent transaction.
  - Graph/subgraph-level prediction, e.g. uncovering a money-laundering network.



Designing graph features

- The key requirement for successful node, edge and graph-level predictions
  - Expressive, transferable and generalizable features
- Type of raw input features
  - Intrinsic/structural features : node features s.a. node degree, link features e.g. shortest distance between node pair, graph features s.a. bag-of-graph patterns.
  - Extrinsic data features : node features s.a. atom type, edge features e.g. bond type, graph features s.a. molecular energy.
- Development timeline
  - Before 2014 : Engineered/hand-crafted features, e.g. pageRank
  - 2014-2016 : Shallow learned features, e.g. node2vec
  - After 2016 : Deep learned features, e.g. GNNs

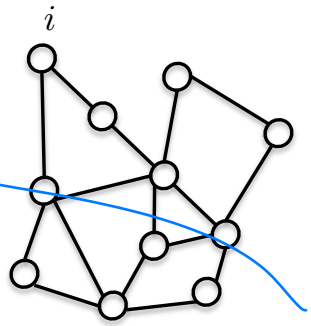


Outline

- Graph prediction tasks
- **Graph feature without learning (engineered)**
  - **Node features**
  - Link features
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

Node-level features

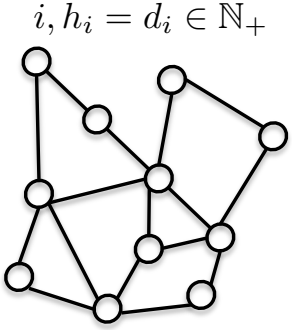
- There exist two key properties to characterize nodes in a graph.
- Structural Information : Inherent node characteristics within the graph structure, e.g. node degree.
- Positional Information : Spatial location or orientation, s.a. 3-dimensional Euclidean coordinates of atoms.



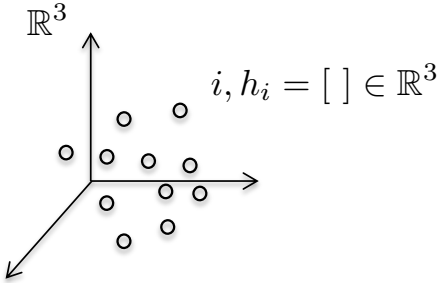
$\mathcal{G}$

Node feature

$\Rightarrow$



Structural information
e.g. node degree



Positional information
e.g. Euclidean coordinates

For what ?
∵ Graphs have no notion of directions
But if we express it in the Euclidean space, we do have directions
⇓
Good because directions are very important for expressivity

transform graph to Euclidean space

Intrinsic features (eg. node degree)

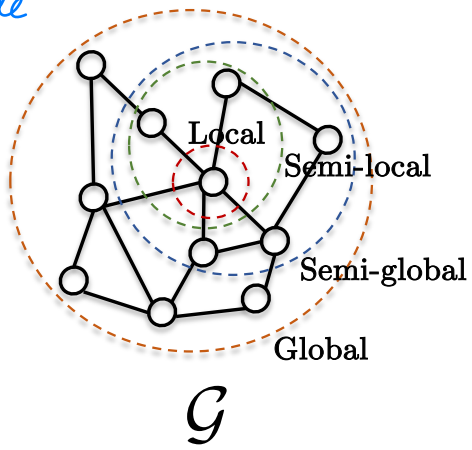
represent graph by its eigenvectors $v_i \in \mathbb{R}^n$
⇒ The graph is represented by all its eigenvectors in the Euclidean space

Intrinsic Structural node features

- Define the node features from graph topology (structure).
  - Local properties of graphs
    - E.g. node degree : The number of neighboring nodes directly connected to a node.
  - Semi-local properties
    - E.g. graphlet degree : Rich description of a node's local neighborhood (later discussed)
  - Semi-global properties
    - E.g. betweenness centrality : Measure of a node's importance in facilitating paths (later discussed)
  - Global property of graphs
    - E.g. eigenvector centrality s.a pageRank : Overall influence of a node in the entire network (discussed previously)

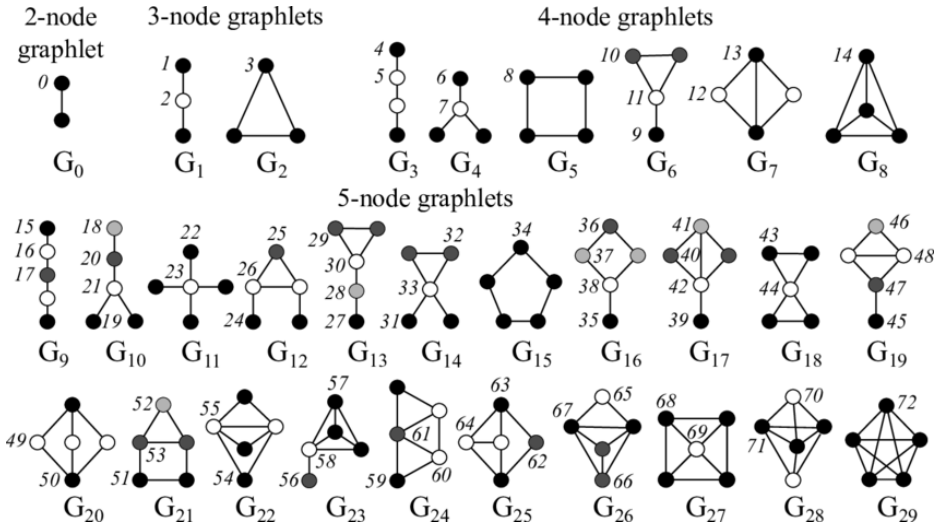
How many shortest paths go through each node

Eigenvectors of Laplacian or Adjacency matrix

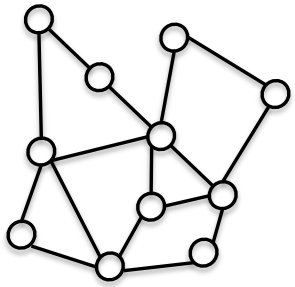


Graphlets

- Graphlets[1], a.k.a. network motifs, serve as fundamental building blocks or patterns within a graph.
- The number of graphlets increases with the number of nodes that compose them.
- Any graph can be represented by the number and types of graphlets it contains.



Source[2]



$\mathcal{G}$

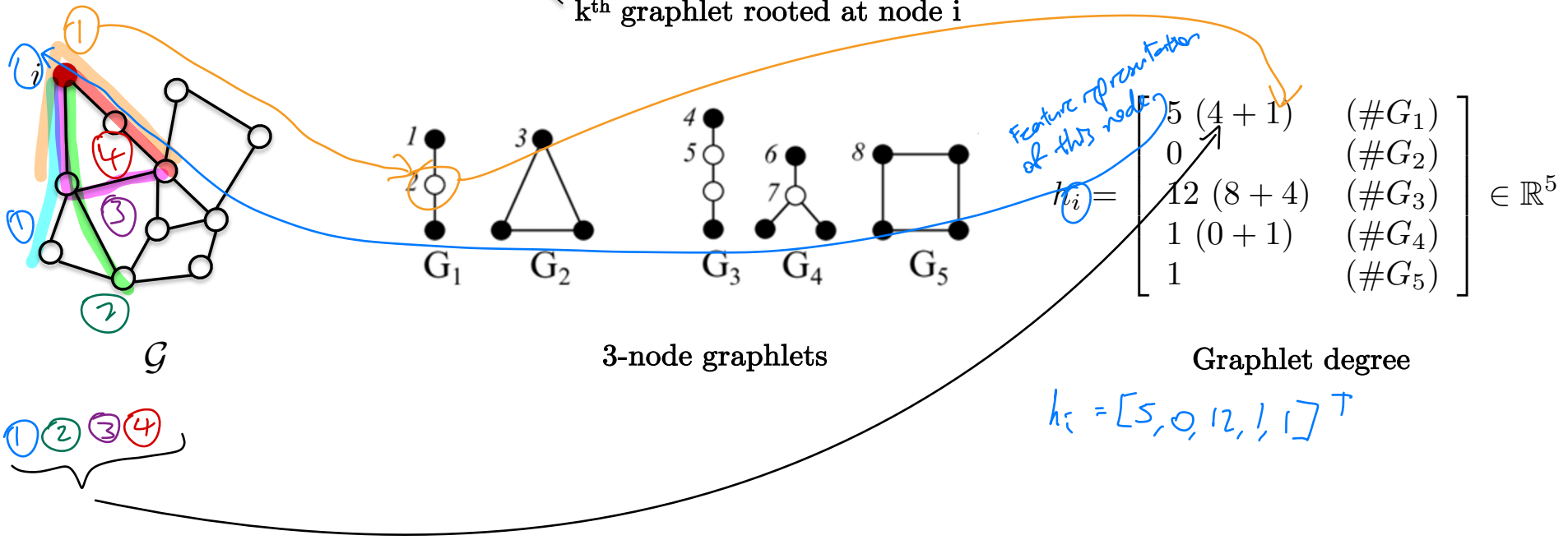
[1] Przulj, Corneil, Jurisica, Modeling Interactome, Scale-Free or Geometric, 2004
[2] Yaveroglu et-al, Graphlets: A Package for ERG Modeling Based on Graphlet Statistics, 2014

Graphlets

- Graphlets degree : A vector representing the number of graphlets rooted at a node.
- It provides a complete and detailed characterization of the local neighborhood topology around that node.

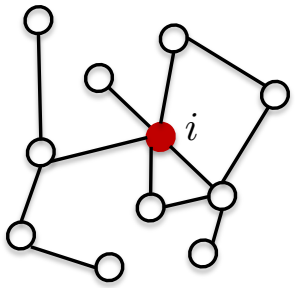
$$h_{i,k} = \#(g_k(i) \subset G) \text{ for } k = 1, ..., d = n_k$$

k^{th} graphlet rooted at node i

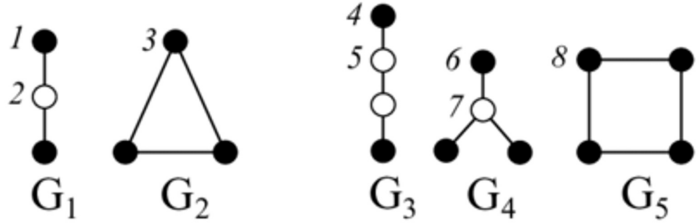


In-lecture question

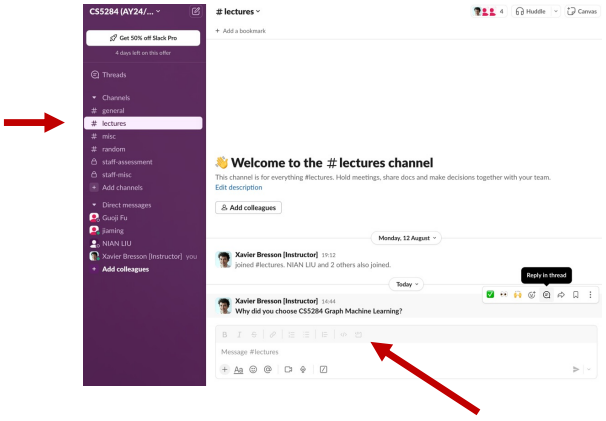
- What is the graphlet degree vector for the red node i?
- In Slack #lectures
  - Identify the question and Reply in thread with a short response
- Answer :



$\mathcal{G}$



3-node graphlets



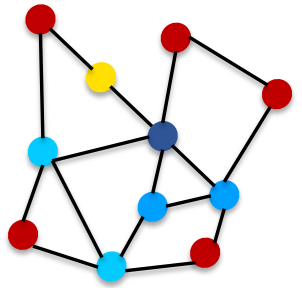
Betweenness centrality

- **Betweenness centrality[1] : This measures the number of shortest paths that pass through a given node.**

$$h_i = \sum_{s \neq i \neq t} \frac{\sigma_{\mathrm{st}}(i)}{\sigma_{\mathrm{st}}} \in \mathbb{R}_+$$

where σ_{st} is the number of shortest paths from node s to node t, and $\sigma_{\mathrm{st}}(i)$ is the number of thos paths that pass though i.

Interpretation: A node with a high betweenness centrality is considered more significant, as more information or interactions are likely to flow through it.



$\mathcal{G}$

Color coding value of betweenness centrality (dark red represents low values and dark blue indicates high values)

[1] Freeman, A set of measures of centrality based on betweenness, 1977

PageRank

Eigenvector centrality

Centrality value of each node is just the corresponding index value of

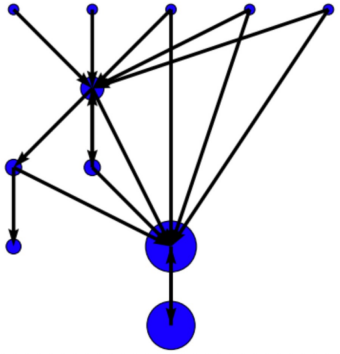
$A[\,] =$ centrality of node 1 / centrality of node N

- Global property of a graph
  - A measure of a node's influence within the network.
  - High score indicates that a node is connected to many others and thus is influent.
  - Various types of spectral centrality measures exist.
  - A well-known example is PageRank[1], which is based on solving a spectral problem.
    - According to the Perron-Frobenius Theorem[2,3], eigenvector centrality is defined as the stationary state of the graph. It is represented by the largest eigenvector (corresponding to eigenvalue 1) of the adjacency matrix.

$$Ah = h \in \mathbb{R}^N, \; h_i \in \mathbb{R}_+$$

Adjacency matrix of the graph

Largest eigenvector/ eigenvector centrality



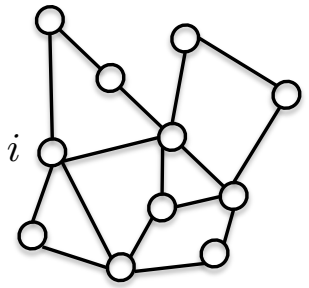
[1] Page, Brin, Motwani, Winograd, The PageRank citation ranking: Bringing order to the web, 1999
[2] Perron, Zur Theorie der Matrices, 1907
[3] Frobenius, Ueber Matrizen aus nicht negativen Elementen, 1912

Positional node feature

Laplacian PE

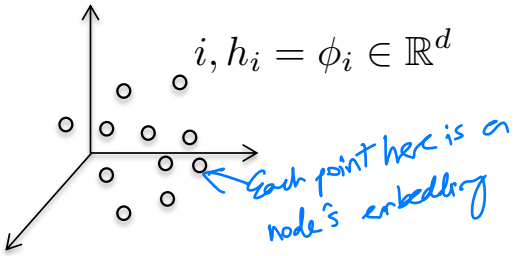
- Geometric features : Raw input features s.a. 3-D coordinates or angles of atoms.
- Topological/structural positional features : Positional features derived directly from the structure of the graph.
  - Manifold learning : Techniques like Laplacian eigenvectors[1] embed the graph into a d-dimensional Euclidean space while preserving the node proximity in the graph topology.

$$L = \mathrm{I} - D^{-1/2} A D^{-1/2} = \Phi^T \Lambda \Phi \in \mathbb{R}^{n \times n}$$

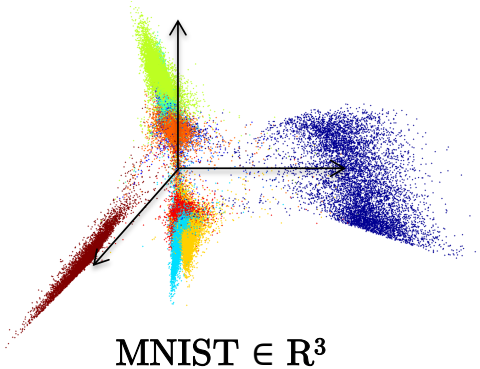


$\mathcal{G}$

$\Rightarrow$



Euclidean embedding of the graph with Laplacian eigenvectors



MNIST $\in \mathbb{R}^3$

[1] Belkin, Niyogi, Laplacian eigenmaps for dimensionality reduction and data representation, 2003

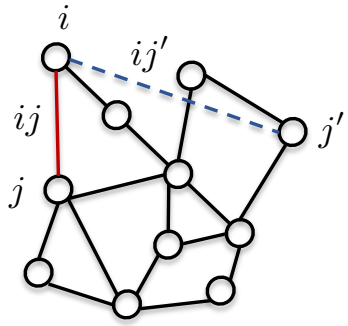
☆ to read

Outline

- Graph prediction tasks
- **Graph feature without learning (engineered)**
  - Node features
  - **Link features**
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

Link-level features

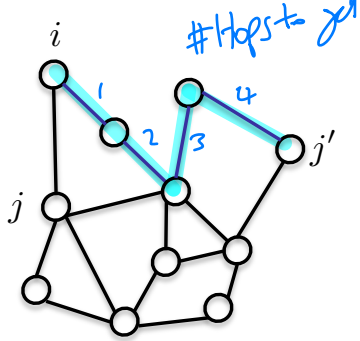
- Edges/links are defined by pairs of nodes.



$\mathcal{G}$

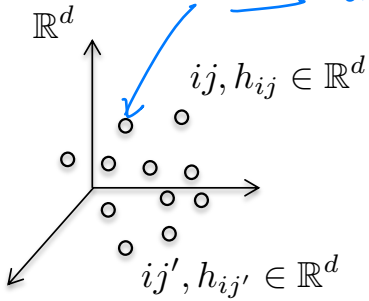
Edge/link features

$\Rightarrow$



#Hops to get to j' from $i=4$

Structural information
e.g. shortest path
between (i,j) or (i,j')



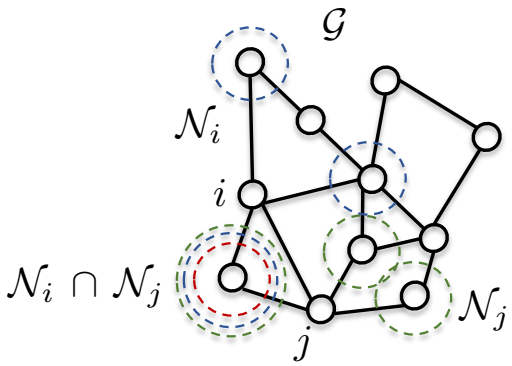
Each point here is an EDGE unlike before

Positional information
e.g. Euclidean
embedding of edges

Edge-level features

- Edge features derived from graph topology :
  - Local properties of graphs
    - Jaccard index[1] : Measures the similarity between two nodes by calculating the proportion of shared neighbors to the total number of neighbors.

$$h_{ij} = \frac{\mathcal{N}_i \cap \mathcal{N}_j}{\mathcal{N}_i \cup \mathcal{N}_j} \in \mathbb{R}_+$$



$$h_{ij} = \frac{1}{5}$$

  - Semi-global properties of graphs (next slides)
    - Katz index
    - Shortest path
    - Diffusion distance

[1] Jaccard, The Distribution of the Flora in the Alpine Zone, 1912

Katz index

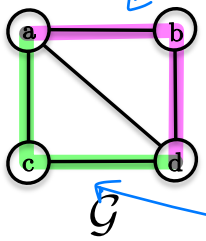
works on directed graph (?)

- Katz index[1] quantifies the number of walks between two nodes in a graph.
- The number of walks of length k between two nodes i and j is given by the k-th power of the adjacency matrix A^k, where each entry $A_{ij} \in \{0,1\}$ represents whether an edge exists between nodes i and j.

* proof (?)

$$h_{ij} = A^k_{ij} \in \mathbb{N}_+$$

2 ways to get to d from a in 2 hops



$\mathcal{G}$

$$A = A^1 = \begin{array}{c} \begin{matrix} a & b & c & d \end{matrix} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{array} \begin{matrix} \text{(node index)} \\ a \\ b \\ c \\ d \end{matrix}$$

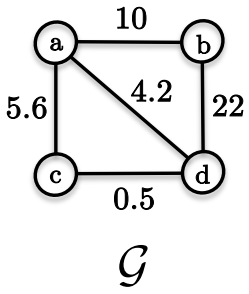
$$A^2 = A^1.A^1 = \begin{array}{c} \begin{matrix} a & b & c & d \end{matrix} \\ \begin{bmatrix} 3 & 1 & 1 & 2 \\ 1 & 2 & 2 & 1 \\ 1 & 2 & 2 & 1 \\ 2 & 1 & 1 & 3 \end{bmatrix} \end{array} \begin{matrix} a \\ b \\ c \\ d \end{matrix}$$

[1] Katz, A New Status Index Derived from Sociometric Analysis, 1953

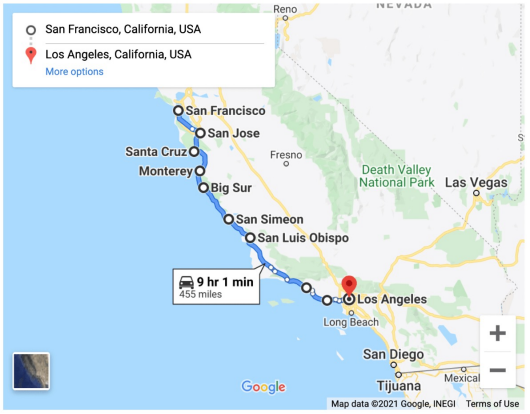
useful in the context of molecular graphs, but don't know why

Shortest path

- The shortest path between two nodes in a network serves as a robust edge feature.
- Dijkstra's algorithm[1] (using Fibonacci heap min-priority queue) has a linear complexity O(E) when computing the shortest distances from any real-valued adjacency matrix.



$\mathcal{G}$

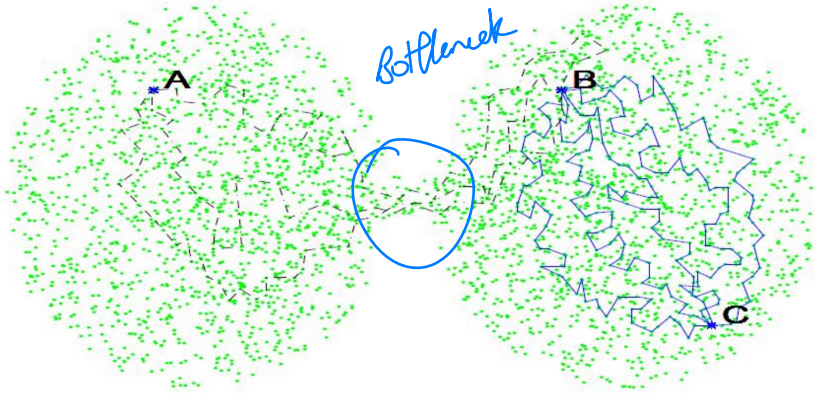


[1] Dijkstra, A note on two problems in connexion with graphs, 1959

Good for social networks ⭐ Important

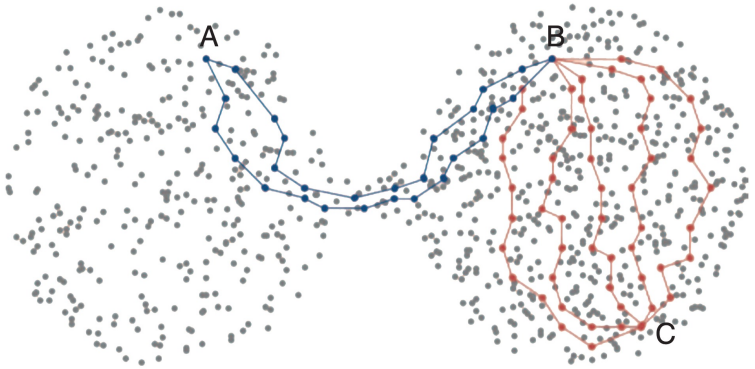
Diffusion distances

- The diffusion distance between two nodes in a network also represents informative edge feature.
  - Random walk distance[1]
  - Heat/Laplacian diffusion distance[2]



$d_{AB}^{\text{Diffusion}} \gg d_{BC}^{\text{Diffusion}}$ ← How to calculate (?)

Diffusion distance[3]



$d_{AB}^{\text{Shortest}} \approx d_{BC}^{\text{Shortest}}$

Shortest path[4]

Note: This is a good example where diffusion dist is better than shortest paths:
shortest path A ↔ B ≅ shortest path B ↔ C
But A & B are blocked by a bottleneck
↳ we want our edge feature to reflect this
↳ Diffusion dist does it for us.

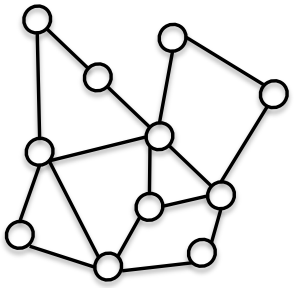
[1] Page, Brin, Motwani, Winograd, The PageRank citation ranking: Bringing order to the web, 1999
[2] Kondor and Vert, Diffusion kernels, 2004
[3] Rotbart, Diffusion maps, 2012
[4] Talmon, Cohen, Gannot, Coifman, Diffusion Maps for Signal Processing, 2013

Outline

- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

Graph-level features

- Define a feature vector that encapsulates the entire graph structure, a.k.a. graph signature.
- Kernel techniques can be used to establish bag-of-graph features and calculate distances between two graphs. Notable examples are
  - Graphlet kernels[1]
  - Weisfeiler-Lehman kernels[2]



$\mathcal{G}$

Graph feature representation

$\Rightarrow$

$$h_{\mathcal{G}} = \begin{bmatrix} \\ \\ \end{bmatrix} \in \mathbb{R}^d$$

[1] Shervashidze, Vishwanathan, Petri, Mehlhorn, Borgwardt, Efficient graphlet kernels for large graph comparison, 2009
[2] Shervashidze, Schweitzer, Van Leeuwen, Mehlhorn, Borgwardt, Weisfeiler-lehman graph kernels, 2011

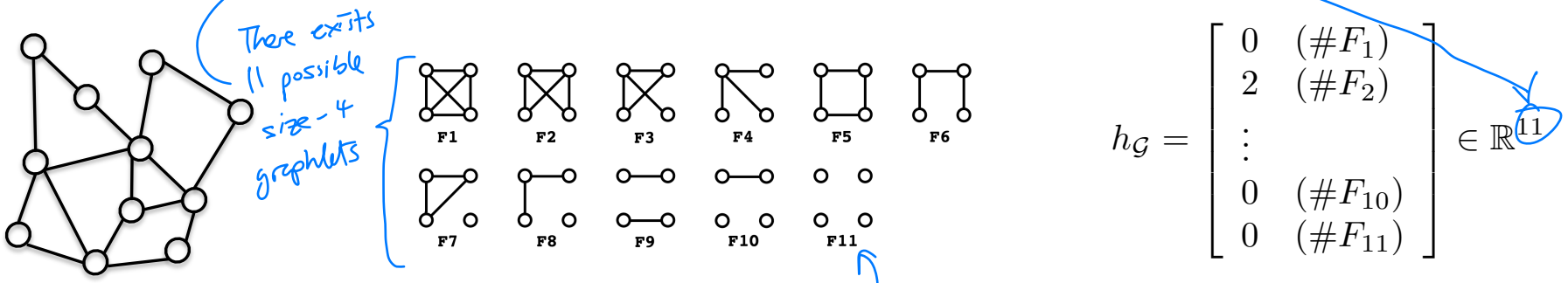
Graphlets

- Given a collection of size-k graphlets, compute the number of each graphlet within a graph, often referred to as the bag-of-graphlets.

$$h_{\mathcal{G}_p} = \#(g_p \subset \mathcal{G}) \text{ for } p = 1, ..., d = n_k$$

cardinality

size-k graphlets



$\mathcal{G}$

All size-4 graphlets[1]
Note that graphlets do not need to be connected (i.e. isolated nodes)

$$h_{\mathcal{G}} = \begin{bmatrix} 0 & (\#F_1) \\ 2 & (\#F_2) \\ \vdots & \\ 0 & (\#F_{10}) \\ 0 & (\#F_{11}) \end{bmatrix} \in \mathbb{R}^{11}$$

Graph signature

[1] Shervashidze, Vishwanathan, Petri, Mehlhorn, Borgwardt, Efficient graphlet kernels for large graph comparison, 2009

Graphlets

- Given two graphs represented by their graphlet signature vectors, the kernel distance between them is defined as[1] :

dot product

$$K(\mathcal{G}, \mathcal{G}') = \langle h_{\mathcal{G}}, h_{\mathcal{G}'} \rangle \in \mathbb{N}_+$$

- Computational bottleneck : Counting size-k graphlets in a graph of size n has an exponential complexity of $O(n^k)$, making this method impractical for large values of k.

Large time complexity

combinatorial problem

and large n

[1] Shervashidze, Vishwanathan, Petri, Mehlhorn, Borgwardt, Efficient graphlet kernels for large graph comparison, 2009

why does this converge ?

Weisfeiler-Lehman

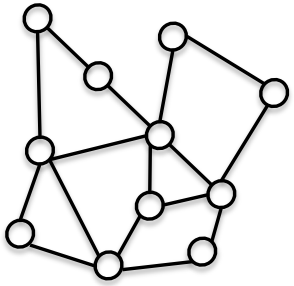
- The Weisfeiler-Lehman graph coloring process[1,2] is a widely used technique for graph signatures and distance measurement.
- This simple iterative coloring procedure is applied to a graph, continuing until no new color can be generated.
- The resulting graph is then represented by a bag of colors, capturing its structural characteristics.

color of node i at iteration k

colors of all the neighbours of node i at iteration $k+1$

$$c_i^{k+1} = f_{\text{coloring}}(c_i^k, \{c_j^{k+1}\}_{j \in \mathcal{N}_i}) \in \mathbb{R}$$
$$= f_{\text{Hash}}(c_i^k, \{c_j^{k+1}\}_{j \in \mathcal{N}_i}) \in \mathbb{R}$$
$$h_{\mathcal{G}_c} = \#(i = c) \text{ for } c = 1, ..., d = n_{\text{color}}$$

usually implemented via a hashing function or a hashtable



$\mathcal{G}$

$$h_{\mathcal{G}} = \begin{bmatrix} 1 & (\#c_1) \\ 3 & (\#c_2) \\ \vdots & \\ 2 & (\#c_d) \end{bmatrix} \in \mathbb{R}^d$$

Histogram ("bags") of colors

↓

signature of graphs

[1] Weisfeiler, Lehman, A reduction of a graph to a canonical form and an algebra arising during this reduction, 1968
[2] Shervashidze, Schweitzer, Van Leeuwen, Mehlhorn, Borgwardt, Weisfeiler-lehman graph kernels, 2011

was used a lot in biology

Weisfeiler-Lehman

- An example[1]

$$c_i^{k+1} = f_{\text{coloring}}(c_i^k, \{c_j^{k+1}\}_{j \in \mathcal{N}_i}) \in \mathbb{R}$$
$$= f_{\text{Hash}}(c_i^k, \{c_j^{k+1}\}_{j \in \mathcal{N}_i}) \in \mathbb{R}$$
$$h_{\mathcal{G}_c} = \#(i = c) \text{ for } c = 1, ..., d = n_{\text{color}}$$

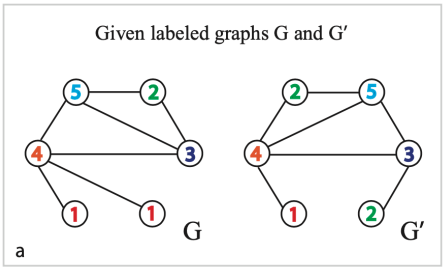
will differ if both graphs' topology differ

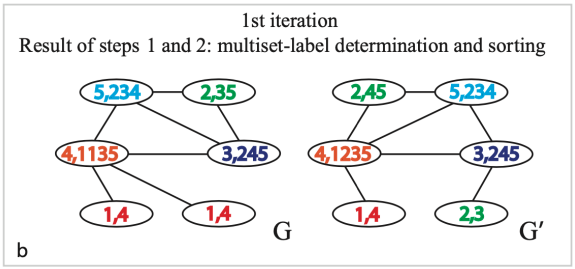
$h_G = (2, 1, 1, 1, 1, 2, 0, 1, 0, 1, 1, 0, 1)$

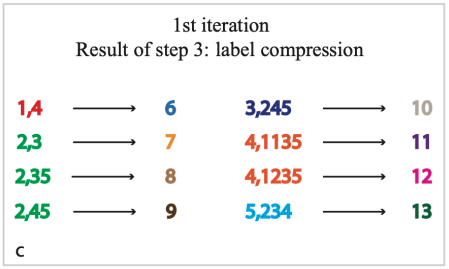
$h_{G'} = (1, 2, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1)$

1,2,3,4,5,6,7,8,9,10,11,12,13

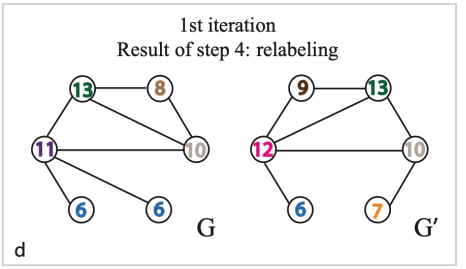
Dictionary of created colors







Hash table



[1] Shervashidze, Schweitzer, Van Leeuwen, Mehlhorn, Borgwardt, Weisfeiler-lehman graph kernels, 2011

Weisfeiler-Lehman

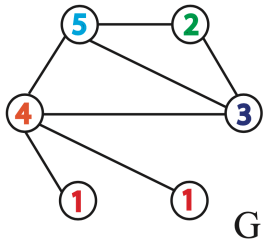
- Given two graphs, the WL kernel distance[1] is defined as

$$K(\mathcal{G}, \mathcal{G}') = \langle h_{\mathcal{G}}, h_{\mathcal{G}'} \rangle \in \mathbb{N}_+$$

dot product

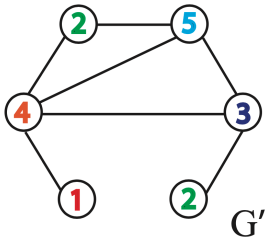
- The complexity of this process is linear O(E) as it involves coloring all nodes and their neighbors.

very good. Best today in terms of time



G

$h_G = (2, 1, 1, 1, 1, 2, 0, 1, 0, 1, 1, 0, 1)$



G′

$h_{G'} = (1, 2, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1)$

$$K(\mathcal{G}, \mathcal{G}') = \langle h_{\mathcal{G}}, h_{\mathcal{G}'} \rangle = 11$$

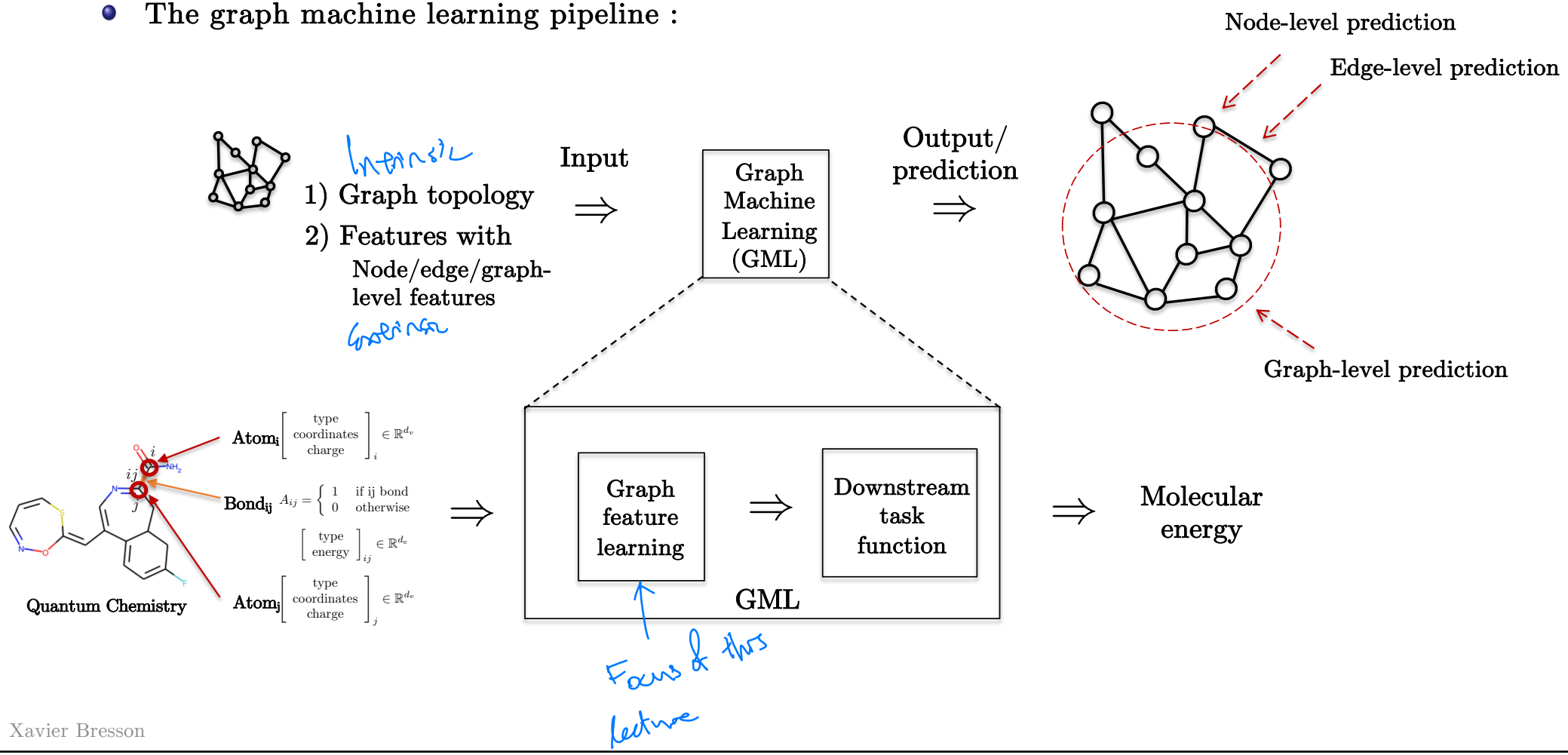
[1] Shervashidze, Schweitzer, Van Leeuwen, Mehlhorn, Borgwardt, Weisfeiler-lehman graph kernels, 2011

Outline

- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- **Shallow graph feature learning**
  - **Node features**
  - Link features
  - Graph features
- Conclusion

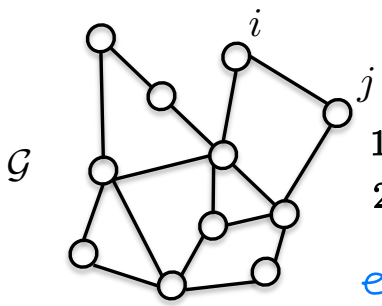
Graph feature learning

- The graph machine learning pipeline :



Node feature learning

- Let us focus on the node feature learning task :

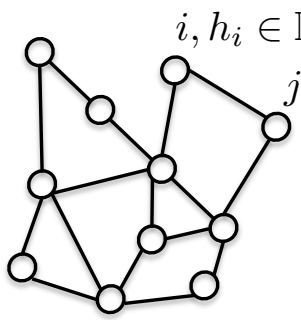


1) Graph topology
2) Node-level feature

Input $\Rightarrow$

Node feature learning (multiple layers)

Node encoding $\Rightarrow$



Downstream task $\Rightarrow$

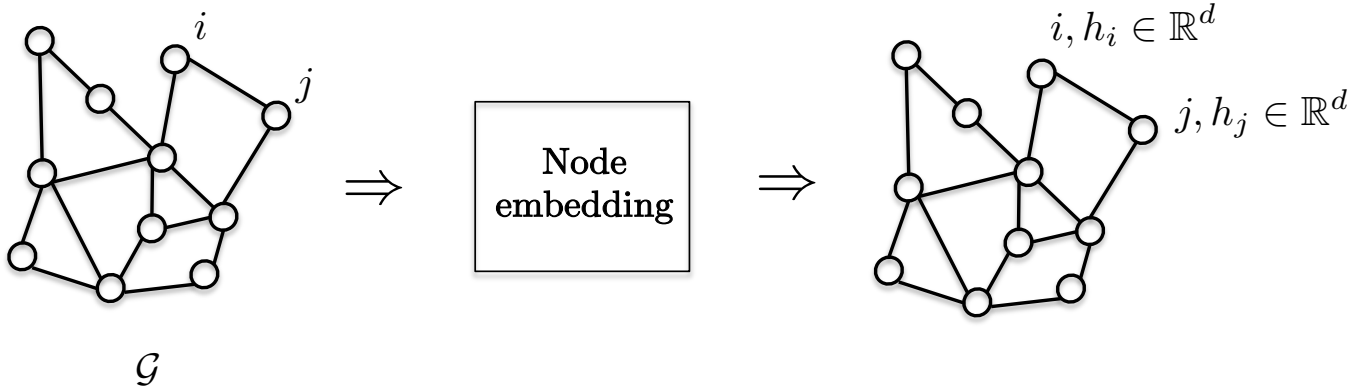
e.g. 3D coords / atom type / orientation / etc.

Not used in the past. We also ignore it for this lecture.

Only work with graph topology for shallow learning

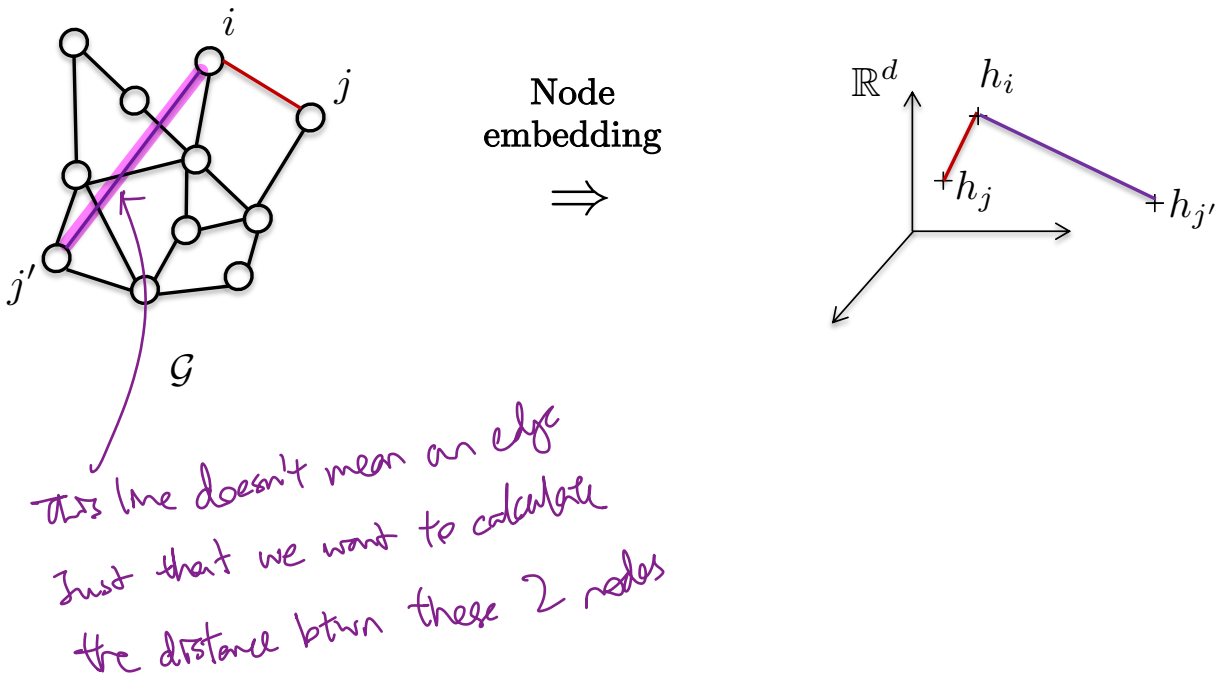
Shallow node feature learning

- Hypothesis
  - Shallow learning : Single hidden layer, a.k.a. node embedding or encoding.
  - Relies exclusively on the adjacency matrix to capture graph topology.
    - No extrinsic node features are used s.a. atom type.
- How do we compute $h_i \in R^d$?



Fundamental embedding property

- The embedding space must preserve the proximity of nodes on the graph.
- If a pair of nodes (i,j) are close together on the graph G, their corresponding vectors (h_i,h_j) in R^d should also be close in the embedding space.



used more in practice :: & speed

Embedding similarity

- The similarity between two vectors can be defined using
  - Euclidean distance : $s_{ij} = ||h_i - h_j||_2 \in R_+$
  - Scalar product : $s_{ij} = \langle h_i, h_j \rangle$ (or $h_i^T h_j$) $\in R$ ← fast in GPU
  - Computational complexity for both similarities is O(d).
  - Note that Euclidean distance equals scalar product for L^2-normalized vectors i.e. $||h_i||_2 = 1$.
- Other distances can be used s.a. optimal transport/Wasserstein distance[1] but the computational complexity is usually higher, often at least $O(d^2)$.

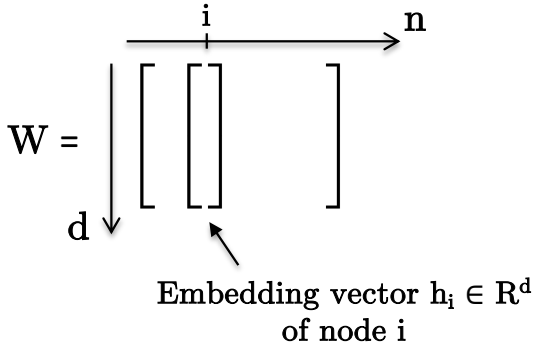
more theoretically sound

But $O(d^2)$

[1] Cuturi, Sinkhorn distances: Lightspeed computation of optimal transport, 2013

Shallow embedding

- The most straightforward node feature learning is as follows :
  - One-hot encoding : Each node index $i \in \{0,1,...,n-1\}$ is represented as a one-hot encoding vector $1_i = [0,0,..0,1,0,...,0] \in R^n$.
  - Linear Transformation : This one-hot vector is then linearly transformed to produce the node embedding vector $h_i = W1_i \in R^d$, with $W \in R^{d \times n}$.
- Interpretation : The matrix W serves as a learnable lookup table, allowing the model to map node indices to meaningful embeddings.



Shallow embedding

- How do we compute or learn the parameters W?
- Standard learning technique
  - Select a Similarity Metric : Choose a similarity distance measure to evaluate the similarity between two node embeddings.
  - Design a Loss Function : Create a loss function that promotes embedding similarity based on the selected metric.
  - Optimize Parameters : Use gradient descent optimization to find the optimal parameters W that minimize the loss function.

mostly stochastic GD

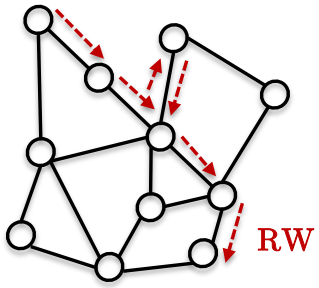
Applies to sequences of words/tokens

But there is no notion of sequence in graphs

∴ We extract RWs on graphs that are analogous to sequences

Node-to-vector

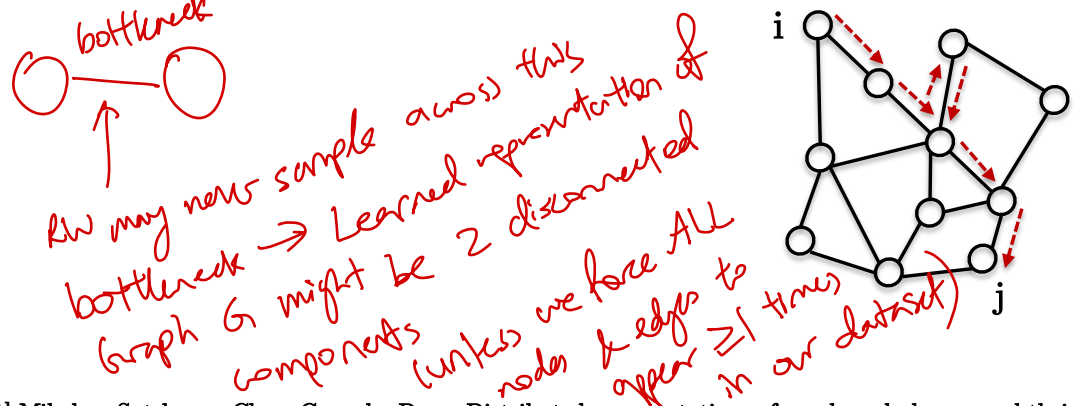
- Motivation is to extend word2vec[1] from NLP to graphs.
  - DeepWalk[2] and node2vec[3]
- Key idea :
  - Use random walks (RWs) to extract sequences of nodes (equivalent to sequence of words with word2vec).
  - This process helps explore the graph topology.



[1] Mikolov, Sutskever, Chen, Corrado, Dean, Distributed representations of words and phrases and their compositionality, 2013
[2] Perozzi, Al-Rfou, Skiena, Deepwalk: Online learning of social representations, 2014
[3] Grover, Leskovec, node2vec: Scalable feature learning for networks, 2016

Random walks

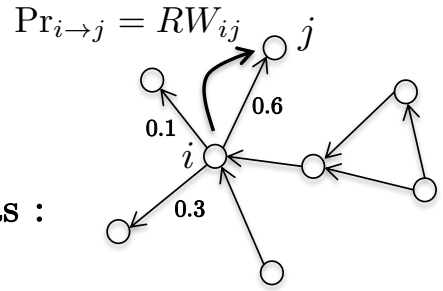
- The exploration is stochastic, which is an advantage and a limitation.
- Advantage : RWs are fast to sample and can be done in parallel.
- Limitation : A large number of RWs may be needed to explore the entire graph, especially since they are biased as they tend to focus on local neighborhoods.
- Overall, RWs identify nodes that are closely connected in the graph.
- We will assume that two nodes i and j are close if they belong to the same RW.



RW starting at i and visiting j after a fews steps/hops.

[1] Mikolov, Sutskever, Chen, Corrado, Dean, Distributed representations of words and phrases and their compositionality, 2013
[2] Perozzi, Al-Rfou, Skiena, Deepwalk: Online learning of social representations, 2014
[3] Grover, Leskovec, node2vec: Scalable feature learning for networks, 2016

Random walks

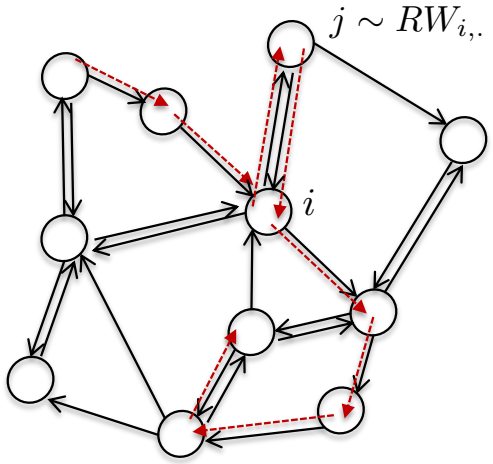


- How to generate a RW?
  - We start by defining the random walk probability transition matrix as :

$$RW = D^{-1}A, \quad D_{ii} = \sum_j A_{ij}$$

  - Each row of RW, i.e. $RW_{i,.}$, corresponds to the probability to move from vertex i to vertex j.
  - A RW path is obtained by iteratively sampling the RW matrix with a Bernouilli distribution : Given a current node i, we sample the next node j from the distribution $j \sim RW_{i,.}$. After sampling, we update i to be j and repeat the process.

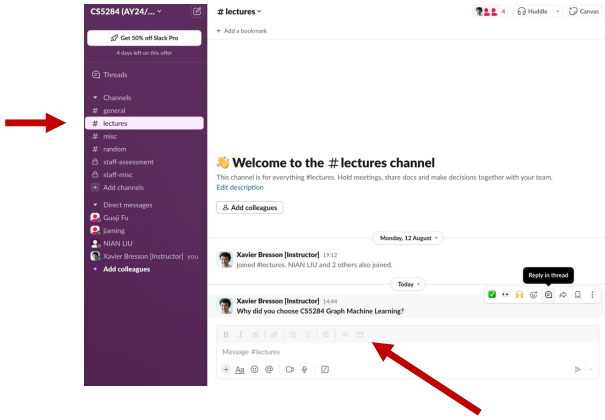
categorical



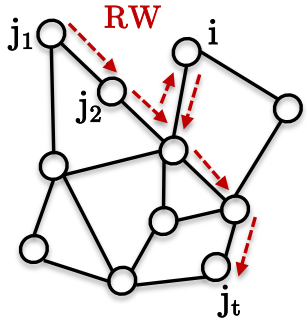
Sampling a RW path (⇢) in a graph

In-lecture question

- Provide an alternative to random walks to extract graph's topological information?
- In Slack #lectures
  - Identify the question and Reply in thread with a short response
- Answer :



Probability of RW



- How to model the probability of a RW, namely $P_{RW}(j_1,j_2,...,j_t)$?
- Skip-gram model :
  - Using the chain rule for probabilities :

$$P_{\mathrm{RW}}(j_1, j_2, ..., j_t) = \Pi_{i \in \mathrm{RW}} P(j_1|j_2, j_3, ..., i) P(j_2|j_3, ..., i) ... P(j_t|i) P(i)$$

  - Suppose each node j only depends on node i (i.e. independent of other nodes) :

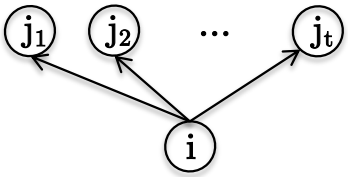
$$P_{\mathrm{RW}}(j_1, j_2, ..., j_t) = \Pi_{i \in \mathrm{RW}} P(j_1|i) P(j_2|i) ... P(j_t|i) P(i)$$

Skipgram assumption.

  - This is equivalent to assume that each node i (the center word) can predict the other nodes j (the context words) in the RW, which leads to the formulation P(j|i) :

$$P_{\mathrm{RW}}(j_1, j_2, ..., j_t) = \Pi_{i \in \mathrm{RW}} P(i) \ \Pi_{j \in \mathrm{RW} \setminus i} \ P(j|i)$$

Probability of observing this RW sequence



Probability of graph topology

- **Probability of predicting a RW:**

$$P_{\mathrm{RW}}(j_1, j_2, ..., j_t) = \Pi_{i \in \mathrm{RW}} P(i)\ \Pi_{j \in \mathrm{RW} \setminus i} P(j|i)$$

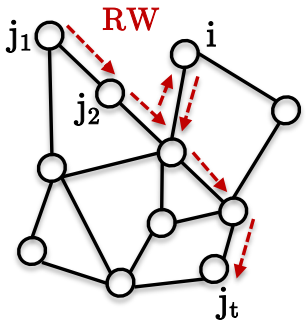
- **Probability of predicting entire graph topology can be expressed as:**

$$P_{\mathrm{G}} = \mathbb{E}_{\mathrm{RW}}\Big(P_{\mathrm{RW}}(j_1, j_2, ..., j_t)\Big) \leftarrow \text{over all RWs}$$

$$= \mathbb{E}_{\mathrm{RW}}\Big(\Pi_{i \in \mathrm{RW}} P(i)\ \Pi_{j \in \mathrm{RW} \setminus i} P(j|i)\Big)$$

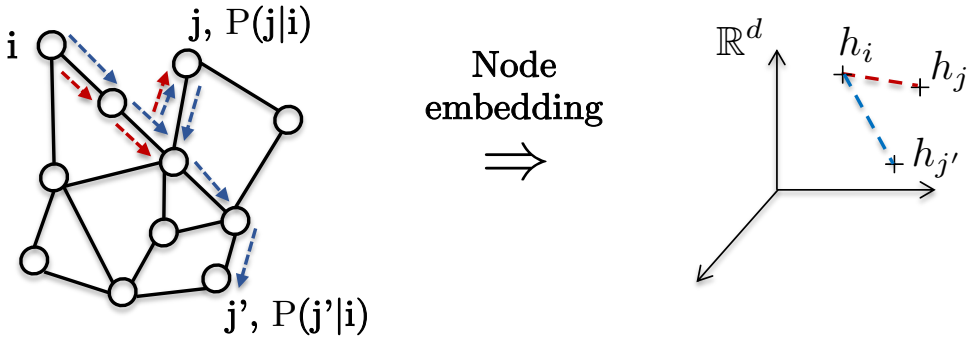
Expectation over all possible RWs

Constant/uniform probability (all nodes are equally important), P(i) can be dropped.



Parametrizing conditional probability

- How to parametrize the conditional P(j|i) w.r.t. node embeddings h_i , $h_j \in R^d$?
  - If nodes i and j are close on the graph (or equivalently in a RW), then their embeddings (h_i,h_j) should also be close in the embedding space R^d.
  - Equivalently, if nodes i and j are close on the graph (or in a RW), we should have high probability P(j|i).
  - Conclusion : High probability P(j|i) implies similar embeddings (h_i,h_j), and inversely.



Parametrizing conditional probability

- Different designs for P(j|i) are possible :

$$P(j|i) = f_W(h_i, h_j) = f_W(W1_i, W1_j)$$

$$= \frac{\sigma(d_{ij})}{Z}$$

$$\sigma(\cdot) = \begin{cases} \exp(\cdot) \\ \text{sigmoid}(\cdot) \end{cases}$$

Base function
(sparse softmax, dense softmax)

$$d_{ij} = \begin{cases} h_i^T h_j \\ \|h_i - h_j\|_2^2 \end{cases}$$

Similarity score
(dot product, Euclidean distance)

$$Z = \sum_{j' \in V} \sigma(d_{ij'})$$

Normalization constant
(partition function)

normalizing constant to make P(j|i) a valid prob dist

$$h_\eta = W1_\eta, \eta = i, j$$

Embedding of node η with
learnable lookup table W
(computed by optimization)

Node embedding matrix

1-hot encoding vector

TLDR: $\max_W P_G \equiv \min_W -\log P_G$

Loss function

- Our goal is to maximize the probability P_G derived from all random walks (RWs), or equivalently minimize the negative log probability -log P_G (leveraging the log function for faster and more stable computations) :

$$\max_W \mathbb{E}_{\text{RW}}\Big(\Pi_{i\in\text{RW}}\Pi_{j\in\text{RW}\setminus i}P(j|i)\Big)$$

$$\Leftrightarrow \min_W L(W) = \mathbb{E}_{\text{RW}}\Big(-\sum_{i\in\text{RW}}\sum_{j\in\text{RW}\setminus i}\log P(j|i)\Big)$$

- Minimizing this loss function L encourages all node embeddings h_j associated with the RWs to be close to the node embedding h_i.
- However, estimating this loss presents a computational challenge, as the number of all possible RWs is substantial, rendering the direct calculation intractable.

Approximation

- We need to approximate the probability P_G :

$$P_G = \mathbb{E}_{RW}\left(\Pi_{i \in RW} \Pi_{j \in RW \setminus i} P(j|i)\right)$$

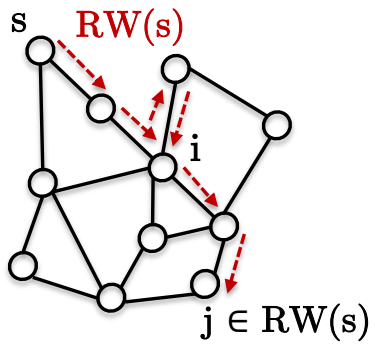
- Approximation strategy
  - Starting Nodes: Consider all nodes s ∈ V in the graph as starting nodes for random walks.
  - RW Generation: For each starting node s, generate a single fixed-length random walk RW(s) (fast parallel execution).
  - Skip-gram assumption: Given the random walk RW(s), each node within this walk can be treated as a center node i that predicts the other nodes j in the same walk :

$$P_G \approx \Pi_{s \in V} \Pi_{i \in RW(s)} \Pi_{j \in RW(s) \setminus i} P(j|i)$$

  - Objective: Minimize the negative log probability -log P_G :

$$\min_W L(W) = -\sum_{s \in V} \sum_{i \in RW(s)} \sum_{j \in RW(s) \setminus i} \log P(j|i)$$

Loss function



Algorithm

- Two-step algorithm
  - Random Walk Sampling : For each node s in the graph, sample a random walk RW(s) in parallel for all nodes.
  - Loss Minimization : Minimize the likelihood probability loss associated with the generated random walks :

$$
\begin{aligned}
L(W) &= -\sum_{s\in V}\sum_{i\in \mathrm{RW}(s)}\sum_{j\in \mathrm{RW}(s)\setminus i}\log P(j|i) \\
&= -\sum_{s\in V}\sum_{i\in \mathrm{RW}(s)}\sum_{j\in \mathrm{RW}(s)\setminus i}\log \frac{\sigma(d_{ij})}{Z} \\
&= -\sum_{s\in V}\sum_{i\in \mathrm{RW}(s)}\sum_{j\in \mathrm{RW}(s)\setminus i}\log \frac{\sigma(d_{ij})}{\sum_{j'\in V}\sigma(d_{ij'})} \\
&= -\sum_{s\in V}\sum_{i\in \mathrm{RW}(s)}\sum_{j\in \mathrm{RW}(s)\setminus i}\log \frac{\exp(h_i^T h_j)}{\sum_{j'\in V}\exp(h_i^T h_{j'})}, h_\eta = W1_\eta, \eta = i, j, j'
\end{aligned}
$$

with $\sigma = \exp$ and $d_{ij} = h_i^T h_j$

2 for loops

Still too expensive to optimize as complexity is $O(n^2)$!

Negative sampling

- Fast and popular technique for removing the partition function Z (denominator) in the probability calculation[1].
  - Step 1 : Approximate the original problem by considering nodes outside the RW path :

$$\max_W \; L(W) = \log \Pi_{s \in V} \Pi_{i \in \mathrm{RW}(s)} \Pi_{j \in \mathrm{RW}(s) \setminus i} P(j|i)$$

$$\Downarrow \quad \text{approximate}$$

$$\max_W \; L_{\mathrm{NS}}(W) = \log \Pi_{s \in V} \Pi_{i \in \mathrm{RW}(s)} \Pi_{j \in \mathrm{RW}(s) \setminus i} \underbrace{\Pi_{k \notin \mathrm{RW}(s)}} \frac{P(j|i)}{P(k|i)}$$

Positive samples

Negative samples

Can be approximated with uniform random distribution over the nodes, $k \sim U_V$

  - Interpretation : The original probability of positive sample is reinforced by the probability of negative sample (i.e. maximizing ratio $P_{pos}/P_{neg} \Rightarrow P_{pos} > P_{neg}$).

[1] Mikolov, Sutskever, Chen, Corrado, Dean, Distributed representations of words and phrases and their compositionality, 2013

Negative sampling[1]

- **Step 2 : Change the log product into the sum of log :**

$$\max_W \; L_{\text{NS}}(W) = \log \Pi_{s\in V} \Pi_{i\in \text{RW}(s)} \Pi_{j\in \text{RW}(s)\setminus i} \Pi_{k\sim U_V} \frac{P_{\text{RW}}(j|i)}{P_{\text{RW}}(k|i)}$$

$$= \sum_{s\in V} \sum_{i\in \text{RW}(s)} \sum_{j\in \text{RW}(s)\setminus i} \sum_{k\sim U_V} \log \frac{P(j|i)}{P(k|i)}$$

$$\text{with } P(j|i) = \frac{\sigma(d_{ij})}{Z}, P(k|i) = \frac{\sigma(d_{ik})}{Z}$$

$$= \sum_{s\in V} \sum_{i\in \text{RW}(s)} \sum_{j\in \text{RW}(s)\setminus i} \sum_{k\sim U_V} \log \sigma(d_{ij}) - \cancel{\log Z} - \log \sigma(d_{ik}) + \cancel{\log Z}$$

No more Z !

$$= \sum_{s\in V} \sum_{i\in \text{RW}(s)} \sum_{j\in \text{RW}(s)\setminus i} \sum_{k\sim U_V} \log \sigma(d_{ij}) - \log \sigma(d_{ik})$$

$$= \sum_{s\in V} \sum_{i\in \text{RW}(s)} \sum_{j\in \text{RW}(s)\setminus i} \sum_{k\sim U_V} \log \sigma(h_i^T h_j) - \log \sigma(h_i^T h_k), h_\eta = W 1_\eta, \eta = i, j, k$$

with $d_{ij} = h_i^T h_j$

[1] Mikolov, Sutskever, Chen, Corrado, Dean, Distributed representations of words and phrases and their compositionality, 2013

Algorithm

- Two-step algorithm
  - For each node s in the graph, sample a random walk RW(s) in parallel.
  - Minimize the likelihood probability loss using gradient descent :

$$\min_W \; L_{\mathrm{NS}}(W) = -\sum_{s\in V} \sum_{i\in \mathrm{RW}(s)} \sum_{j\in \mathrm{RW}(s)\setminus i} \sum_{k\sim U_V} \log \sigma(h_i^T h_j) - \log \sigma(h_i^T h_k), h_\eta = W 1_\eta, \eta = i, j, k$$

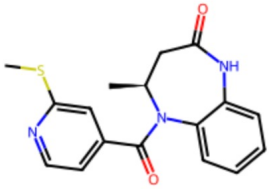
with σ = sigmoid

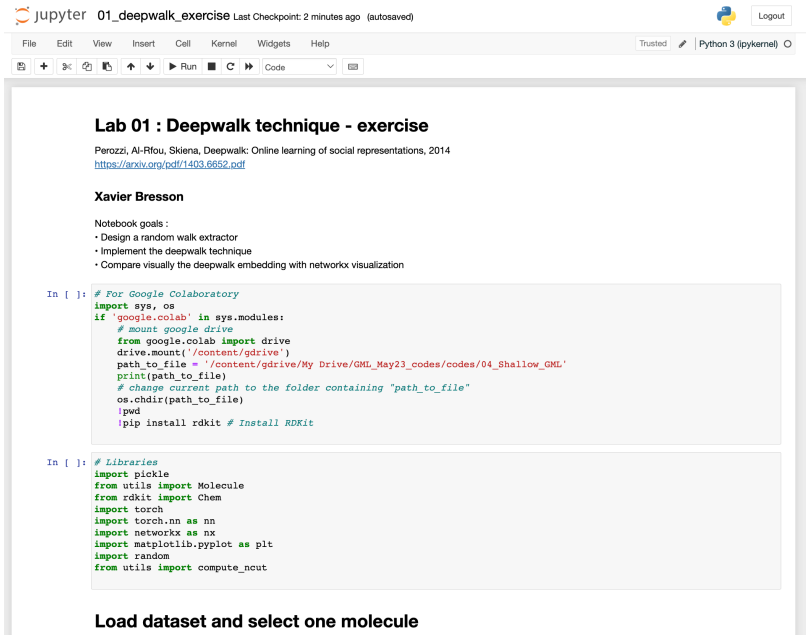
no need for-loop anymore like before to calculate Z

  - The overall complexity of this algorithm is linear, O(n).

Lab 1 : DeepWalk

- Implement DeepWalk[1] (2024 KDD Test of Time Award)





jupyter 01_deepwalk_exercise Last Checkpoint: 2 minutes ago (autosaved)

File Edit View Insert Cell Kernel Widgets Help — Trusted — Python 3 (ipykernel)

Lab 01 : Deepwalk technique - exercise

Perozzi, Al-Rfou, Skiena, Deepwalk: Online learning of social representations, 2014
https://arxiv.org/pdf/1403.6652.pdf

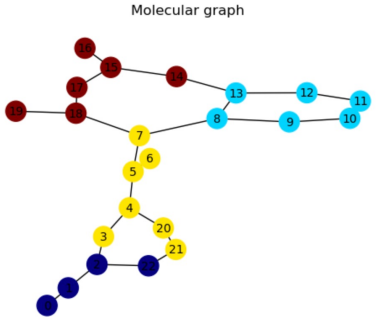
Xavier Bresson

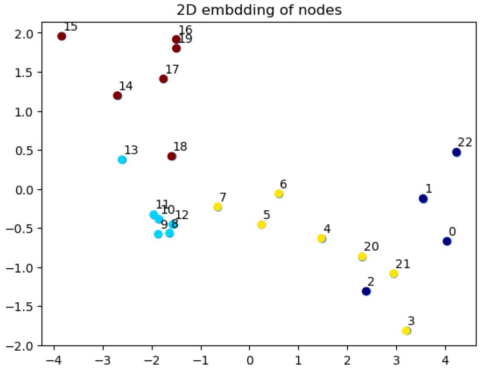
Notebook goals :
- Design a random walk extractor
- Implement the deepwalk technique
- Compare visually the deepwalk embedding with networkx visualization

In [ ]:
```
# For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/My Drive/GML_May23_codes/codes/04_Shallow_GML'
    print(path_to_file)
    # change current path to the folder containing "path_to_file"
    os.chdir(path_to_file)
    !pwd
    !pip install rdkit # Install RDKit
```

In [ ]:
```
# Libraries
import pickle
from utils import Molecule
from rdkit import Chem
import torch
import torch.nn as nn
import networkx as nx
import matplotlib.pyplot as plt
import random
from utils import compute_ncut
```

Load dataset and select one molecule





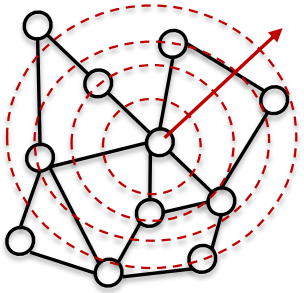
[1] Perozzi, Al-Rfou, Skiena, Deepwalk: Online learning of social representations, 2014

Node2vec

- Random walks (RWs) sample the graph's geometry without any guidance.
- While this approach is mathematically sound, it requires a large number of RWs to accurately approximate the original loss :

$$L(W) = \mathbb{E}_{\mathrm{RW}}\Big(- \sum_{i \in \mathrm{RW}} \sum_{j \in \mathrm{RW} \setminus i} \log P(j|i) \Big)$$

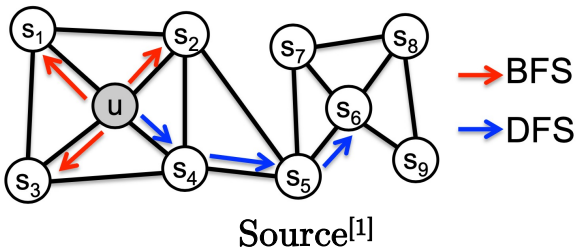
- To enhance the stochastic exploration of the graph's topology, we can improve the generation of RWs by initially focusing on local exploration and subsequently expanding to a global perspective[1].



[1] Grover, Leskovec, node2vec: Scalable feature learning for networks, 2016

Controlled RW path

- We can control a new RW with three explorative actions :
  - Local exploration with BFS (breadth-first search) to explore nearby nodes.
  - Global exploration with DFS (depth-first search) to traverse deeper into the graph.
  - No exploration (move back to the previous node).

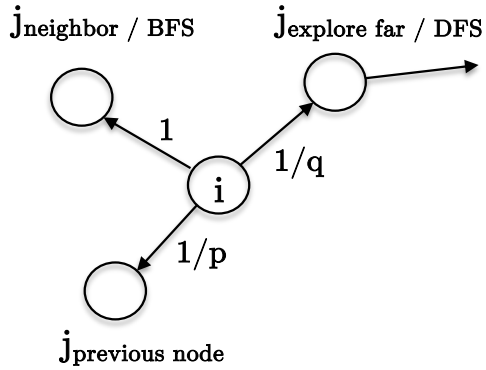


Source[1]

- To sample the next action, we introduce two parameters :
  - A parameter q to balance the trade-off between BFS and DFS.
  - A parameter p to determine the likelihood of returning to the previous node.
  - Probability of actions sampled by Bernoulli :

$$P_{\text{action}}(j|i) = \frac{1}{c} \begin{bmatrix} 1/p \\ 1 \\ 1/q \end{bmatrix} \begin{matrix} \text{Back to previous node} \\ \text{DFS} \\ \text{BFS} \end{matrix}$$

Normalization constant



- This approach maintains a linear complexity of O(n).

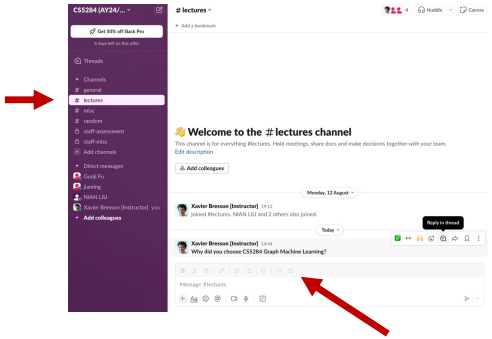
Summary

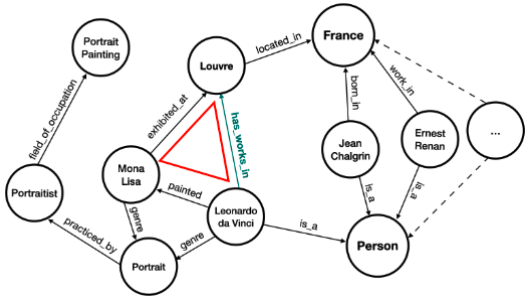
- Learning shallow node-level features
  - Leverage graph's topology : Two nodes are considered similar if they are topologically close on the graph, as determined by a sampled random walk.
  - The choice of node closeness can be tailored to specific tasks.
  - For instances, RWs can be substituted with the shortest path for applications in molecular science or with other diffusion processes like spectral distance.
  - This learning method is shallow, involving only a single hidden layer, which limits the expressiveness of the features.
  - Additionally, only graph topology is used in this approach — no raw features have been incorporated thus far.

W lookup matrix

In-lecture question

- Can you transfer the node embeddings trained on a graph G1 to a new graph G2?
- In Slack #lectures
  - Identify the question and Reply in thread with a short response
- Answer :

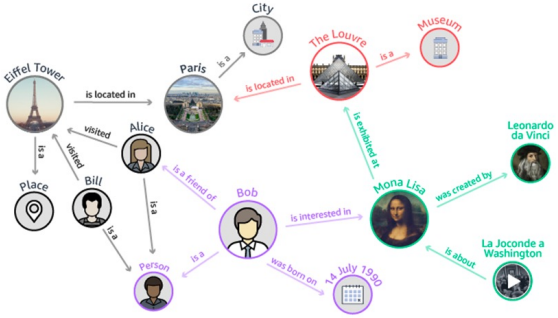




Source[1]

Is transfer learning possible ?

$\Rightarrow$



Source[2]

[1] Hebatallah et-al, Locality-aware subgraphs for inductive link prediction in knowledge graphs, 2023
[2] https://aws.amazon.com/neptune/knowledge-graphs-on-aws

Outline

- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- **Shallow graph feature learning**
  - Node features
  - **Link features**
  - Graph features
- Conclusion

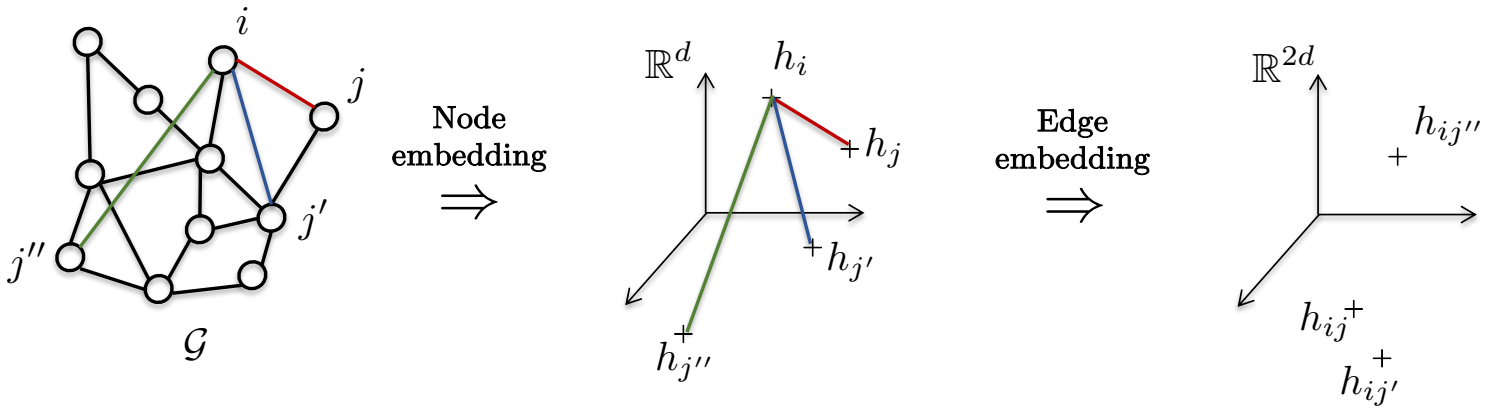
No : If we learnt good node embeddings, $h_i \approx h_j \Rightarrow \therefore \text{concat}(h_i, h_j) \approx \text{concat}(h_j, h_i)$

Link-level features

wouldn't concat (h_i, h_j) or concat (h_j, h_i) make a huge diff ?

- **Link embedding : A straightforward yet effective method for link embedding is to concatenate the embeddings of a pair of nodes.**

$$h_{ij} = \text{Concat}(h_i, h_i) \in \mathbb{R}^{2d}, \quad h_\eta = W_1 1_\eta, \eta = i, j$$



Same as RW sampling before, just now on edges

Link-level features

- Link embeddings can be computed by predicting that the link kl is closely associated with the link ij along the RW path originating from node i.
- This approach establishes connections between links based on their proximity within the graph's structure.

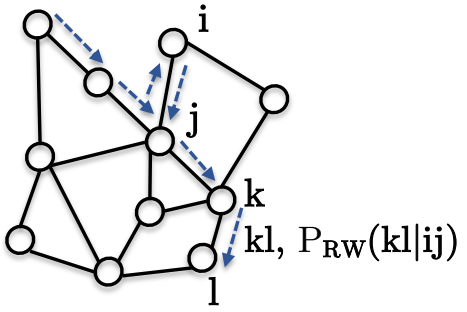
$$L(W) = \mathbb{E}_{\text{RW}}\Big(- \sum_{ij \in \text{RW}} \sum_{kl \in \text{RW} \setminus ij} \log P(kl|ij) \Big)$$

more expressivity

$$h_{\eta\gamma} = W_2 \text{Concat}(W_1 1_\eta, W_1 1_\gamma) \in \mathbb{R}^d, W_1 \in \mathbb{R}^{d \times N}, W_2 \in \mathbb{R}^{d \times 2d}, \eta = i, k, \gamma = j, l$$

$$P(ij|kl) = \frac{\exp(h_{ij}^T h_{kl})}{\sum_{k'l' \in V^2} \exp(h_{ij}^T h_{k'l'})}$$

Does not work very well in practice



Outline

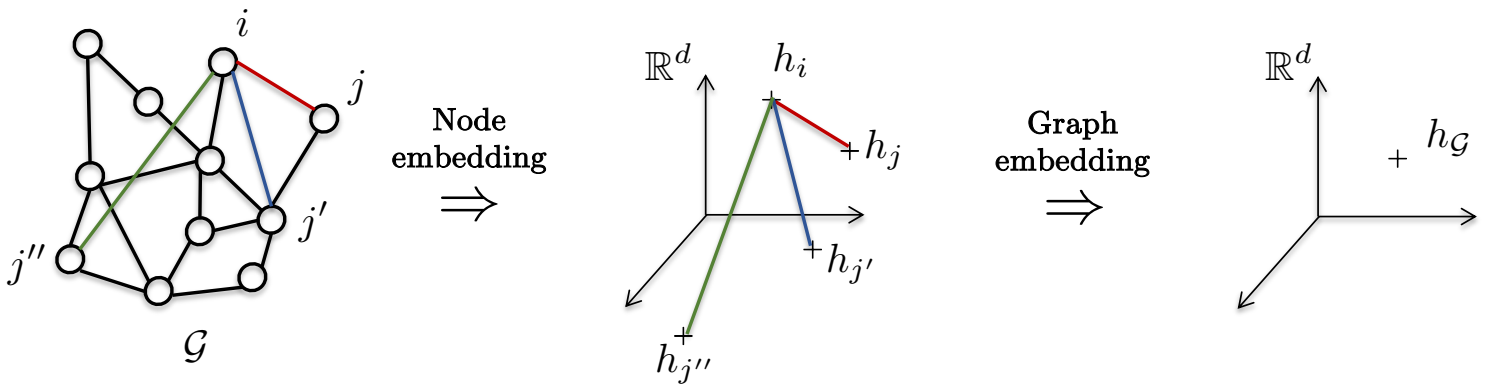
- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- **Shallow graph feature learning**
  - Node features
  - Link features
  - **Graph features** // Not in exam
- Conclusion

Good baseline in practice

Graph-level feature

- **Pre-trained graph embedding : A straightforward yet effective method for graph embedding involves taking the mean or sum of pre-trained node embeddings :**

$$h_{\mathcal{G}} = \frac{1}{N} \sum_{i \in V} h_i \in \mathbb{R}^d$$



Not in exam

Graph-level feature

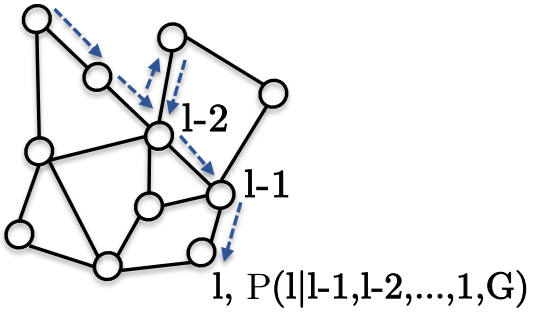
- Learning graph embedding :
  - RW embedding of graphs[1] : This technique computes the graph embedding by predicting RWs of lengths L, L+1,...,L+M, based on the RWs of shorter lengths.
  - This approach captures the structural and relational information of the graph.

wot ?

$$L(W) = \mathbb{E}_{\text{RW}}\Big(- \sum_{l=L}^{L+M} \log P(l|l-1, l-2, ..., 1, G)\Big)$$

with $h_G = w \in \mathbb{R}^d$

Learnable vector
representing the graph embedding



[1] Ivanov, Burnaev, Anonymous walk embeddings, 2018

Outline

- Graph prediction tasks
- Graph feature without learning (engineered)
  - Node features
  - Link features
  - Graph features
- Shallow graph feature learning
  - Node features
  - Link features
  - Graph features
- Conclusion

Conclusion

- Engineered graph features
  - Engineered graph features are limited in expressivity and are often domain-specific, relying heavily on expert knowledge.
- Shallow learned features
  - These features represent an extension of the word2vec model to graphs through pre-trained node embeddings.
  - Limitations
    - Architectural constraints : The use of only one hidden layer restricts the model's expressivity and generalization capabilities -- deeper architectures are generally more effective.
    - Neglecting raw features : There is no integration of raw node, edge or graph features, which means critical information are missing.
    - Lack of transferability : The learned features are not transferable to new graphs, leading to generalization failures in different contexts.

Questions?